

El patrón **MVVM (Model – View – ViewModel)** en Android es una forma de organizar el código para que sea **más fácil de mantener, probar y escalar**.

1. Partes del patrón

- **Model (M):**
Es la **fuentes de datos** y la lógica de negocio.
☞ Aquí van las bases de datos, las APIs, los repositorios.
Ejemplo: una clase que obtiene una lista de usuarios desde internet.
 - **View (V):**
Es lo que el **usuario ve en pantalla** (Activity, Fragment, XML).
☞ Solo muestra información y reacciona a las acciones del usuario.
Ejemplo: un `RecyclerView` que muestra la lista de usuarios.
 - **ViewModel (VM):**
Es el **intermediario** entre el Model y la View.
☞ Contiene la lógica para preparar los datos que la View necesita.
☞ Expone los datos con objetos observables (`LiveData`)
☞ La View **observa** estos datos y se actualiza automáticamente cuando cambian.
-

2. Cómo se comunican

- La **ViewModel** pide datos al **Model**.
 - El **Model** devuelve los datos.
 - La **ViewModel** los prepara y los expone con `LiveData`.
 - La **View** está “suscrita” (observando) y se actualiza sola cuando el dato cambia.
-

3. Ejemplo rápido

Supongamos una app que muestra una lista de productos:

- **Model:** `ProductRepository` que trae productos desde una API.
- **ViewModel:** `ProductViewModel` que tiene un `LiveData<List<Product>>`.
- **View:** `ProductFragment` con un `RecyclerView` que observa ese `LiveData`.

Cuando el `Repository` trae datos → la `ViewModel` los guarda en `LiveData` → la `View` los muestra automáticamente.

Es decir:

- **Model:** de dónde vienen los datos.
- **ViewModel:** prepara y expone los datos.
- **View:** solo los muestra y escucha al usuario.

¿Qué NO debería hacer la View?

- ✗ Consultar directamente la base de datos o la API.
- ✗ Contener lógica de negocio o cálculos pesados.
- ✗ Decidir cómo transformar los datos (eso lo hace el ViewModel).

En Android, la **solicitud de permisos** es un caso especial porque:

- **Solo la View (Activity/Fragment/Compose)** puede interactuar directamente con el **ciclo de vida** y con el **sistema operativo** para pedir permisos.
- El **ViewModel no debería manejar permisos**, porque no tiene acceso al `Context` ni debería depender de APIs de Android directamente.

◆ ¿Por qué los dialogs van en la View?

1. **Dependencia del contexto** → para crear un `Dialog` necesitas un `Context` (Activity/Fragment). El ViewModel no debería depender de eso.
 2. **Ciclo de vida** → el sistema maneja cuándo mostrar/ocultar diálogos, algo propio de la UI.
 3. **Separación de responsabilidades** → el ViewModel se limita a exponer **eventos** (“mostrar mensaje de error”, “confirmar acción”), y la View decide **cómo representarlos** (un Toast, un Snackbar, un Dialog).
-

◆ Patrón recomendado en MVVM

- **ViewModel:** expone un evento (ej: “usuario necesita confirmar algo”).
- **View:** escucha ese evento y muestra un diálogo.

◆ Resumen

- ✓ **ViewModel:** emite eventos → "mostrar confirmación", "mostrar error", "mostrar éxito".
- ✓ **View:** decide cómo mostrarlos (Dialog, Snackbar, Toast, etc.).
- ✗ **Nunca** crear un `Dialog` dentro del ViewModel.